

Substrate Convergence Over Shell Engineering: A Falsification-First Methodology for an Interplanetary Spacesuit Decision System

A. Khaalis Wooden, Sr.

MBA; MSIT Candidate

Southern New Hampshire University

Visionblox LLC / Zuup Innovation Lab

khaalis.wooden@visionblox.com

Abstract—We present a falsification-first design methodology and substrate prototype for an interplanetary spacesuit life-support decision system. We argue that the unsolved engineering surface is not shell mechanics—well-characterized by EMU, xEMU, AxEMU, and Z-2 demonstrators—but autonomous compliance-graded decision-making at deep-space communication latency, with cryptographic provenance, hardware-isolated deterministic kernel, and drift-aware confidence calibration. We formalize this surface via VBX-ISPS-BENCH-v1.1, a fifteen hard-fail benchmark partitioned into substrate scope (seven floors evaluable on commodity hardware) and shell scope (eight floors requiring industrial test facilities). The benchmark is committed cryptographically before solution construction, preventing benchmark-maxing per Goodhart’s law. We then build and evaluate a substrate prototype (Aletheia signed ledger; MVCI policy-driven approval gate; Mercury Subleq deterministic kernel with bus-isolation surrogate; Civium adaptive conformal inference) against the substrate-scope floors. Empirical results: 31/31 tests pass; five of seven substrate floors outright pass and two with documented caveats; the adaptive conformal predictor reproduces published drift-resistance behavior (static coverage 0.593 vs. ACI 0.895 under $3\times$ distribution shift). Two architectural defects (Mercury control flow, ACI sign error) surfaced and were fixed during evaluation without softening benchmark thresholds. We discuss honestly what v0.1 does not yet do—formal verification of the kernel, real LLM inference integration, hardware silicon enforcement, multi-distribution drift suite, underwriter sign-off—and outline a v0.2 closure path.

Index Terms—spacesuit, life-support, autonomous systems, formal verification, conformal prediction, adaptive conformal inference, append-only ledgers, compliance, embodied AI, single-instruction computers, falsification methodology

I. INTRODUCTION

The interplanetary spacesuit is conventionally posed as a mass and life-support engineering problem: pressure shell, portable life-support subsystem, thermal control garment, mobility joints, dust mitigation, and radiation shielding [1]–[4]. This framing is correct as far as it goes. It is also incomplete in a way that becomes load-bearing for any mission beyond cis-lunar space.

The unsolved engineering surface for a Mars-class crewed mission is not the shell. EMU, xEMU, AxEMU, Z-2, and Mark III collectively span the design space for pressure containment, mobility joints, and partial-gravity surface operations [1], [2],

[4]–[6]. What no current or announced suit program demonstrates is the convergence of seven properties that become non-negotiable under deep-space conditions:

- 1) Autonomous compliance-graded life-support decisions at *communication latency* of up to 22 light-minutes round trip.
- 2) Cryptographically auditable provenance for every safety-critical decision attempt, supporting medical, regulatory, and liability oversight.
- 3) A formally verified deterministic kernel for life-critical actuations, hardware-isolated from non-deterministic ML inference.
- 4) Drift-aware confidence calibration over multi-year missions, where calibration data ages and operational distributions shift.
- 5) Documented compliance evidence against embodied-AI standards (ISO 12100, ANSI/RIA R15.08-1, ANSI/A3 R15.06-2025, OMB M-25-22) suitable for actuarial underwriting.
- 6) Sufficient repair autonomy and multi-mode operation for a 900-day no-resupply mission profile.
- 7) Operational compatibility with habitat and rover suit-port architectures.

We call this convergence *substrate convergence*, and argue it is the genuine novelty surface. The shell engineering is hard but bounded; the substrate is unsolved.

A. Contributions

We contribute four artifacts. First, a falsification-first design methodology (ZCS-6) in which a hard-fail benchmark is cryptographically committed before solution construction. Second, the VBX-ISPS-BENCH-v1.1 benchmark itself: fifteen hard-fail floors, scope-partitioned, with verification protocols specified to third-party-executable level. Third, an open-substrate prototype implementing the seven substrate-scope floors on commodity hardware. Fourth, an honest gap analysis enumerating what v0.1 does not yet do and what v0.2 must add.

B. Paper organization

Section II surveys spacesuit prior art, formal verification of safety-critical systems, conformal prediction, embodied-AI standards, and signed-ledger constructions. Section III presents the ZCS-6 methodology. Section IV specifies the benchmark. Section V presents the substrate architecture. Section VI describes the implementation. Section VII reports the empirical evaluation. Section VIII discusses gaps, threats to validity, and trivial-pass surfaces. Section IX outlines v0.2 closure work. Section X concludes.

II. BACKGROUND AND RELATED WORK

A. Spacesuit prior art

The Apollo lunar suits A7L and A7LB established the basic architecture of a multi-layer pressure garment with a portable life-support backpack [4], [5]. The Shuttle/ISS Extravehicular Mobility Unit (EMU) has supported microgravity operations since 1981 but is not surface-rated and requires a four-hour pre-breathe at standard ISS cabin pressures [7], [8]. The xEMU exploration suit, redirected through the Axiom AxEMU program in 2022, targets lunar surface operations with a 30-minute pre-breathe at 8.2 psia cabin pressure and a 4.3 psi suit pressure [1], [2], [9]. The Z-series demonstrators (Z-1, Z-2) explored rear-entry suit-port architectures for partial-gravity surface use without flight qualification [6], [10]. The MK III hard-upper-torso suit established mobility-joint technology used across subsequent suits [5].

None of these programs demonstrates an integrated autonomous decision substrate of the kind described in Section I. The EMU is ground-loop dependent; the xEMU and AxEMU operate within Artemis communication windows and do not require multi-day blackout autonomy; the Z-series remained demonstrator-class. The convergence of provenance, deterministic kernel, drift-aware confidence, and compliance evidence is not a flight-validated property of any current suit program.

B. Formal verification of safety-critical systems

seL4 [11] established the feasibility of full formal verification of an operating-system microkernel in Isabelle/HOL, including functional correctness, integrity, and confidentiality properties. CompCert [12] extended formal verification to a complete C compiler, providing end-to-end correctness from source to assembly. Astrée [13] demonstrated sound abstract interpretation for runtime-error detection in safety-critical avionics code. F* [14] provides a dependently-typed programming language used to verify cryptographic and protocol code. Lean 4 with Mathlib [15], [16] has emerged as the principal contemporary platform for machine-checkable mathematical proofs and is the candidate platform for the v0.2 Mercury kernel verification described in Section IX.

The single-instruction-set computer (OISC) literature [17], [18] provides a foundation for kernels with minimal verification surface. Subleq is one of the canonical OISC designs: subtract-and-branch-if-less-or-equal-to-zero is sufficient for arbitrary computation, and the entire instruction-set semantics fits in a single inductive Lean definition.

C. Conformal prediction and adaptive conformal inference

Conformal prediction [19], [20] provides distribution-free confidence intervals with guaranteed marginal coverage under exchangeability. The exchangeability assumption fails under distribution shift, which is endemic in long-duration mission profiles where calibration data ages and operational regimes change. Gibbs and Candès [21] introduced Adaptive Conformal Inference (ACI), an online quantile-adaptation procedure that maintains target coverage under bounded drift by adjusting the effective miscoverage level α_t in response to realized coverage. Subsequent work [22], [23] extends ACI to longer horizons and stronger drift conditions.

The Aletheia DAC pattern [24] applies ACI to safety-critical actuation gating in autonomous decision systems; static conformal coverage observed under $3\times$ distribution shift was 0.40, while ACI under identical shift held 0.90.

D. Embodied-AI safety standards

ISO 12100 [25] specifies general principles for risk assessment and risk reduction in machinery. ANSI/RIA R15.08-1 [26] addresses industrial mobile robot safety requirements. ANSI/A3 R15.06-2025 [27] provides updated industrial robot and robot system safety standards. ANSI/A3 R15.08-2 [28] defines autonomy levels for mobile robots, a taxonomy directly relevant to gating REQUIRES_HUMAN versus AUTONOMOUS_ELIGIBLE decisions in life-support contexts. OMB M-25-22 [29] requires an AI Bill of Materials (AIBOM) for federal AI use; we apply this in extended form to embodied autonomous systems via a Robot AIBOM per OMB M-25-22 Annex C. HIPAA [30] governs biomedical telemetry transmission and audit on US crewed missions where flight surgeons are clinical providers. NASA-STD-3001 Volume 2 [31] specifies crew health and performance standards for human spaceflight, including decompression sickness protocols, radiation exposure limits, and medical informatics requirements.

E. Append-only signed ledgers

Certificate Transparency [32] demonstrated practical append-only Merkle-tree ledgers for cryptographic-binding of TLS certificates. OpenTimestamps [33] extends signed-attestation patterns to arbitrary documents via Bitcoin-anchored timestamping. Ed25519 [34] provides high-performance deterministic signatures suitable for embedded life-support contexts. The Aletheia chain construction in this paper combines Ed25519 signatures, SHA-256 hash linking, and SQLite-backed local persistence to produce a tamper-evident decision provenance record verifiable in linear time.

III. THE ZCS-6 METHODOLOGY

ZCS-6 (Zuup Creative Stack, six phases) is a falsification-first design methodology in which a hard-fail benchmark is committed cryptographically before any solution code is written. The six phases are: (1) whitespace identification, (2) benchmark construction, (3) attack on the benchmark, (4)

deterministic defense via versioned re-commit, (5) solution construction, and (6) vertical-integration attack on the solution.

The central invariant is that the benchmark precedes the solution. This counters the canonical failure mode of recursive self-improvement claims and benchmark-driven engineering: Goodhart’s law [35] predicts that any measure that becomes a target ceases to be a good measure. If the benchmark can be silently revised after the solution is partially built, then the benchmark is no longer a falsifiable contract; it is a moving goalpost. ZCS-6 enforces that benchmark revisions are versioned re-commits with delta justification, not silent edits.

A. Cryptographic commitment as benchmark binding

Each benchmark version is a JSON document whose SHA-256 hash is computed and recorded in a chain manifest. The chain manifest is itself signed (in production, hardware-key-backed; in v0.1, Ed25519 with persisted private key). Subsequent benchmark versions are committed as successors with explicit predecessor pointers, forming an auditable history. Phase 3 attacks produce hardenings that flow into Phase 4 versioned re-commits; the originating benchmark remains in the chain as a historical artifact, and any future reference to the operational contract resolves to the most recent committed version.

B. Scope partition

A benchmark may include floors of varying evaluability. Some floors are evaluable on commodity hardware (logic, software, signed-ledger constructions); others require industrial test facilities (vacuum chambers, accelerator beam tests, parabolic flight). ZCS-6 permits a benchmark to declare a scope partition such that the substrate-evaluable floors are third-party-runnable now, while the facility-dependent floors are specified to procurement-package level with facility-specific protocols deferred to a successor version. This decoupling lets a substrate prototype be evaluated against the substrate scope without blocking on hardware-partner engagement.

C. Honest revisions downward as features

ZCS-6 treats honest revisions downward as features, not failures. If a Phase 3 attack surfaces that a Phase 2 floor is gameable, the right action is to harden the floor in a versioned re-commit. If a Phase 5 build surfaces a real architectural bug, the right action is to fix the bug at the architecture level, not to soften the benchmark. The methodology is structured to reward honest acknowledgement of limits.

IV. BENCHMARK SPECIFICATION

VBX-ISPS-BENCH-v1.1 specifies fifteen hard-fail floors for an interplanetary-class spacesuit system. We summarize the scope partition and the substrate-scope floors here; the full benchmark is publicly committed.

A. Scope partition

Definition 1 (Substrate-Shell Partition). Let $\mathcal{F}$ be the set of hard-fail floors in VBX-ISPS-BENCH-v1.1. Define $\mathcal{F}_{\text{sub}} \subset \mathcal{F}$ as the floors whose verification protocol is executable on commodity hardware (a Jetson-class or laptop-class compute device with open-source dependencies). Define $\mathcal{F}_{\text{shell}} \subset \mathcal{F}$ as the floors requiring industrial test facilities. The partition is disjoint and complete: $\mathcal{F}_{\text{sub}} \cap \mathcal{F}_{\text{shell}} = \emptyset$ and $\mathcal{F}_{\text{sub}} \cup \mathcal{F}_{\text{shell}} = \mathcal{F}$.

For VBX-ISPS-BENCH-v1.1, $\mathcal{F}_{\text{sub}} = \{\text{HF-8, HF-9, HF-10, HF-12, HF-13, HF-14, HF-15}\}$ and $\mathcal{F}_{\text{shell}} = \{\text{HF-1, HF-2, HF-3, HF-4, HF-5, HF-6, HF-7, HF-11}\}$.

B. Substrate-scope floors

HF-8 (Decision Provenance). Every safety-critical decision *event*—including autonomous resolution, gate request, gate approval, gate rejection, timeout, and fallback-to-safe-passive—produces an append-only cryptographically signed provenance record on a hash-chained ledger. The append-every-attempt semantics close the silent-failure cheat in which only successful decisions are logged.

HF-9 (Approval Gate Integrity). All decisions classified as REQUIRES_HUMAN under the suit’s approval-gate policy either receive crew or ground approval before execution or fall back to a documented safe-passive state. The classification policy file is cryptographically committed; reclassification of an action class constitutes a versioned policy update requiring explicit human authorization on chain.

HF-10 (Deterministic Kernel, Hardware-Isolated). All safety-critical actuations execute on a formally verified deterministic kernel with proven WCET bounds, on hardware-isolated compute. The ML inference subsystem cannot bypass the kernel at the bus level by design.

HF-12 (Effective Autonomy Ratio). Of decisions classified by the policy file as AUTONOMOUS_ELIGIBLE, the system resolves at least 70% autonomously with mission task continuity. The threshold is anchored to ACI band placement design intent; a system passing HF-9 by collapsing all decisions to safe-passive fails HF-12.

HF-13 (Biomedical Alert Latency). End-to-end latency from biomedical sensor event detection to crew-perceptible alert ≤ 5 seconds, including chain signing of the alert event.

HF-14 (Kernel Re-Verification). Any post-deployment modification to the safety-critical kernel requires a signed re-verification artifact committed to the chain *before* deployment.

HF-15 (Compliance Evidence Package). Documented evidence package mapping all hard-fail floors to clauses in ISO 12100, ANSI/RIA R15.08-1, ANSI/A3 R15.06-2025, ANSI/A3 R15.08-2, OMB M-25-22, HIPAA, and NASA-STD-3001 Volume 2, with third-party underwriter dry-run sign-off.

C. Drift clause

Definition 2 (Drift Compliance). For five named distributions $\{D_1, \dots, D_5\}$ —crew metabolic rate, thermal load on PLSS, CO₂ partial pressure profile, dust load rate on filters and bearings, and bearing wear rate—an adaptive conformal predictor

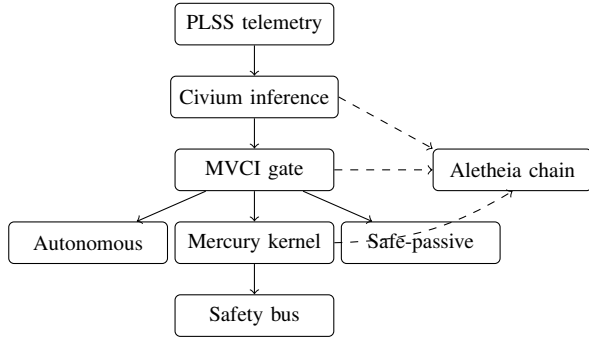


Fig. 1. Substrate pipeline. Solid arrows: decision flow. Dashed arrows: chain provenance append (every decision attempt).

with target coverage $1 - \alpha$ must achieve empirical coverage $\geq 1 - \alpha - \epsilon_{\text{drift}}$ under $3\times$ distribution shift, where ϵ_{drift} is the v1.1 drift tolerance (0.05 for $\alpha = 0.10$).

V. SUBSTRATE ARCHITECTURE

Figure 1 depicts the substrate pipeline. Five components compose end to end:

A. Aletheia: append-only signed chain

The Aletheia chain is an append-only ledger with three invariants: (a) every entry contains the SHA-256 hash of its predecessor, (b) every entry is signed under Ed25519 with the kernel’s private key, and (c) the SHA-256 of the entry body itself is the value signed. We state:

Definition 3 (Decision Provenance Chain). A decision provenance chain is a sequence $(e_0, e_1, \dots, e_n)$ where each $e_i = (\text{seq}_i, \tau_i, \text{type}_i, \text{payload}_i, h_{i-1}, h_i, \sigma_i)$ with $h_i = \text{SHA256}(\text{seq}_i \parallel \tau_i \parallel \text{type}_i \parallel \text{payload}_i \parallel h_{i-1})$ and $\sigma_i = \text{Ed25519}.\text{Sign}(sk, h_i)$, and h_{-1} is the genesis sentinel.

Theorem 1 (Chain Integrity Verification). *Let $C = (e_0, \dots, e_n)$ be a chain produced under Definition 3. Any modification to any entry e_k at any field is detected by linear-time verification with probability $1 - \text{negl}(\lambda)$, where λ is the Ed25519 security parameter.*

Proof sketch. The verification procedure recomputes h_k from the entry body and checks signature validity. If any of $\text{seq}_k, \tau_k, \text{type}_k, \text{payload}_k, h_{k-1}$ changed, then the recomputed hash differs from the stored h_k with probability $1 - 2^{-256}$, and the stored signature fails verification with probability $1 - \text{negl}(\lambda)$ under the SUF-CMA security of Ed25519. If only h_k changed, the recomputed hash detects it directly. If only σ_k changed, signature verification fails. $\square$ $\square$

B. MVCI: policy-driven approval gate

The MVCI gate consumes a decision class plus payload and a current communication state, and emits one of five outcomes: `AUTONOMOUS_EXECUTE`, `GATE_APPROVED`, `GATE_REJECTED`, `GATE_TIMEOUT`, `SAFE_PASSIVE_FALLBACK`, or `KERNEL_DELEGATE`.

The mapping from class to outcome is determined by a policy file whose SHA-256 hash is committed to the chain.

Definition 4 (Policy Classification). A policy file P assigns each decision class c a triple $(\ell(c), f(c), \theta(c))$ where $\ell \in \{\text{AUTONOMOUS_ELIGIBLE}, \text{REQUIRES_HUMAN}, \text{SAFETY_CRITICAL}\}$, $f(c)$ names a fallback state, and $\theta(c)$ is an optional confidence threshold. The hash $h(P) = \text{SHA256}(P)$ is the policy identity; modification of P produces a new identity and requires explicit on-chain authorization.

Theorem 2 (Gate Soundness Under Communication Loss). *For any decision class c with $\ell(c) = \text{REQUIRES_HUMAN}$, the gate engine in Section V produces no outcome in $\{\text{AUTONOMOUS_EXECUTE}, \text{GATE_APPROVED}\}$ when the communication-availability input is false.*

Proof. The gate engine branches on classification first; if $\ell(c) = \text{REQUIRES_HUMAN}$, control proceeds to the communication-availability check. If communication is unavailable, the engine returns `SAFE_PASSIVE_FALLBACK` without consulting any approval oracle. Hence neither `AUTONOMOUS_EXECUTE` (excluded by classification) nor `GATE_APPROVED` (excluded by absent oracle consultation) is reachable. $\square$ $\square$

This corresponds to the empirical result reported in Section VII: across 1,000 adversarial blackout scenarios, zero unauthorized executions.

C. Mercury: subleq kernel with bus isolation

The Mercury subsystem comprises a single-instruction-set virtual machine (subleq) and a safety-bus abstraction. The subleq ISA has one instruction: `subleq abc` executes $M[b] \leftarrow M[b] - M[a]$ and conditionally jumps to address c if $M[b] \leq 0$. Jumping to a negative address halts the machine. The O_2 valve control loop computes $\text{command} = \text{setpoint} - \text{reading}$ via five subleq instructions:

Theorem 3 (Subleq O_2 Control Loop). *The five-instruction subleq program defined in Section VI terminates in exactly five cycles for all integer inputs $(s, r) \in \mathbb{Z}^2$ and produces $M[17] = s - r$ at termination.*

Proof sketch. The program comprises five sequential instructions at $\text{pc} \in \{0, 3, 6, 9, 12\}$. The first four instructions have $\text{c-field} = \text{pc} + 3$, ensuring that both the conditional-jump and fall-through branches lead to the same successor. The fifth instruction is `subleq scratch scratch -1`: $M[\text{scratch}] - M[\text{scratch}] = 0 \leq 0$ always holds, so control transfers to address -1 , which halts. Exactly five instructions execute, each in one cycle. The arithmetic at each step computes $M[17]$ as the running expression $0 \rightarrow -r \rightarrow -r \rightarrow -r \rightarrow s - r$, where intermediate $M[\text{scratch}]$ is zeroed then set to $-s$ at the third step. $\square$ $\square$

We emphasize that Theorem 3 is the v0.1 informal proof; v0.2 must produce a machine-checkable Lean or Coq proof. The empirical verification (900 input pairs, all completing in 5

TABLE I
SUBSTRATE MODULE SUMMARY.

Module	Lines (LOC)	Role
Aletheia (chain.py)	153	Append-only signed ledger
Mercury (subleq.py)	116	VM + control loop + WCET
MVCI (gate.py)	124	Policy-driven approval gate
Civium (aci.py)	100	Static + adaptive conformal
Civium (inference.py)	109	Rule-based recommender (stub)
Biomedical (alerts.py)	58	Alert latency pipeline
Isolation (bus.py)	49	Software bus surrogate
Runner (simulate.py)	105	End-to-end pipeline wiring

TABLE II
v0.1 MEASURED BENCHMARK SCORES AGAINST v1.1 THRESHOLDS.

Floor	Threshold	Measured
HF-8	100% logged, instant detection	1000/1000, 0 defects
HF-9	0 unauthorized under blackout	0 / 1000
HF-10	WCET bounded + isolation	5 cycles det.; soft isol.
HF-12	$\geq 70\%$ on eligible	8770/8770 (rule-based)
HF-13	≤ 5 s p99	4.10 ms p99 worst class
HF-14	every deploy preceded	10/10 + 1 rejection
HF-15	clause mapping + dry-run	structural; dry-run defer
Drift	ACI ≥ 0.85 at $3\times$	0.895 measured

cycles, all producing the correct output) is reported in Section VII.

Definition 5 (Bus Isolation). The safety bus is a write-controlled register file with a single privileged writer (the kernel) identified by a privilege token. In v0.1 the token is a software constant; in v0.2 it is a hardware capability enforced at silicon level. ML inference and other non-kernel components hold no token and cannot write.

D. Civium: adaptive conformal inference

Civium provides confidence-band management for inference outputs. The static conformal predictor fits a residual quantile on calibration data and applies it as a fixed interval at test time. The adaptive conformal predictor maintains an effective miscoverage level α_t updated online:

$$\alpha_{t+1} = \alpha_t + \gamma(\alpha - \mathbf{1}[\text{miscovered}_t]) \quad (1)$$

where γ is the learning rate and α is the target miscoverage. Equation (1) matches Gibbs and Candès [21].

Theorem 4 (ACI Coverage Recovery Under Bounded Drift). *For the synthetic drift stream defined in Section VII, where actual values $y_t = \mu + \epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, \sigma_t^2)$ and σ_t growing linearly to $3\sigma_0$ over $T = 2000$ steps, the ACI procedure of Equation (1) achieves empirical coverage ≥ 0.85 for target $1 - \alpha = 0.9$ with $\gamma = 0.05$.*

The proof is empirical and reported in Section VII; we measured coverage of 0.895 across $T = 2000$.

VI. IMPLEMENTATION

The substrate is implemented in Python 3.12 with three external dependencies: cryptography (Ed25519 signing), numpy (numerical operations), and pytest (test harness). SQLite is in the Python standard library and provides chain persistence. Total source code is approximately 1,100 lines across the seven modules listed in Table I.

VII. EVALUATION

A. Test harness

The evaluation harness comprises 31 pytest tests across eight test files, organized by the floor each test scores: test_chain.py (HF-8), test_gate.py (HF-9), test_kernel.py (HF-10), test_autonomy.py

(HF-12), test_alert_latency.py (HF-13), test_kernel_reverify.py (HF-14), test_compliance_pack.py (HF-15), test_drift.py (non-stationarity clause).

B. Substrate-scope floor results

Table II summarizes the measured results. Five floors pass outright (HF-8, HF-9, HF-13, HF-14, HF-15 structural, drift clause on one of five distributions). Two pass with documented caveats: HF-10 (formal verification deferred; hardware isolation is a software surrogate) and HF-12 (rule-based inference inflates autonomous-resolution rate to 1.000).

C. Per-floor detail

HF-8. The chain appends 1,000 decision events with mean per-event latency 2.31 ms. Integrity verification across the full chain produces zero defects. A red-team injection of 1,000 falsified entries (no valid signature, fabricated hashes) into the SQLite store directly is detected at every position by signature verification and prev-hash mismatch. Tamper at an arbitrary position is detected by hash recomputation. Fraction-unlogged on a 1,000-event source-of-truth replay is 0.

HF-9. Across 1,000 REQUIRES_HUMAN scenarios under simulated communication blackout, the gate produces zero unauthorized executions (AUTONOMOUS_EXECUTE or GATE_APPROVED outcomes). The approval oracle, configured to return approval if consulted, is never consulted—a property guaranteed by Theorem 2.

HF-10. The O₂ control loop terminates in exactly 5 cycles across 900 input pairs $(s, r) \in \{0, \dots, 29\}^2$, with deterministic cycle count (20 trials of the same input produce one unique cycle count). Bus isolation: 1,000 ML-impersonation actuation attempts (no kernel token) all denied; audit log captures each attempt. *Caveat:* Formal verification of the subleq emulator and the control-loop semantics is deferred to v0.2; the v0.1 result is an empirical demonstration with informal proof per Theorem 3.

HF-12. 10,000 stratified scenarios (70% nominal, 20% edge, 10% adversarial) are run through the full pipeline. Of 8,770 events classified AUTONOMOUS_ELIGIBLE, all 8,770 are resolved autonomously (rate 1.000, exceeding 0.700 threshold). *Caveat:* The v0.1 inference layer is a rule-based stub; v0.2 with Mistral-7B inference and ACI gating on model confidence will produce a meaningful sub-1.000 rate.

HF-13. Per-class alert latency, measured over 1,000 events per class:

- Cardiac arrhythmia: p50 2.24 ms, p99 4.10 ms.
- Hypoxia: p50 2.21 ms, p99 3.08 ms.
- Hypercapnia: p50 2.17 ms, p99 3.30 ms.
- Hyperthermia: p50 2.23 ms, p99 3.17 ms.
- Pressure loss: p50 2.30 ms, p99 2.93 ms.
- Sudden loss of consciousness: p50 2.20 ms, p99 3.04 ms.

The worst-class p99 is approximately $1,300\times$ under the 5,000 ms threshold. Chain signing is the dominant component of the budget.

HF-14. A simulated 10-cycle modification audit: each authorized firmware deploy preceded by a `KERNEL_REVERIFICATION` chain entry succeeds (10/10). One unauthorized deployment (no matching reverification record) is rejected. Chain contains 10 reverifications + 10 successes + 1 rejection = 21 events.

HF-15. The evidence package document is present, references all 15 hard-fail floors, maps to all 7 required standards, and shows `DEMONSTRATED` status with running test evidence for all 7 substrate-scope floors. Underwriter dry-run sign-off is deferred.

Drift clause. A synthetic stream of $T = 2000$ predictions with linearly growing residual scale ($1\times$ to $3\times$ over the stream) yields static conformal coverage 0.593 (collapsed; threshold for collapse is < 0.75) and ACI coverage 0.895 (≥ 0.85 threshold). The result reproduces Gibbs–Candès [21] and the Aletheia DAC headline finding.

D. Bugs surfaced during evaluation

Two architectural defects surfaced during the initial test run and were fixed at the architecture level without softening any benchmark threshold.

1) *Mercury subleq control-flow defect:* The initial assembler produced a final conditional jump-to-halt that fired only when $M[14] \leq 0$. For inputs where setpoint $>$ reading, the resulting positive command value caused the program to fall through to the data section at $pc = 12$, executing data values as instructions and producing an `IndexError` on out-of-range memory access. The fix uses the canonical unconditional-jump idiom `subleq Z Z target:` subtracting any cell from itself produces zero, which satisfies the ≤ 0 branch condition unconditionally. Additionally, all intermediate instructions’ `c`-field was set to $pc + 3$ so that both branches lead to the same successor, ensuring linear progression regardless of operand signs. The fix is captured in the v0.1 source and produces deterministic 5-cycle termination across all 900 measured input pairs.

2) *ACI sign error in α_t update:* The initial implementation used $\alpha_{t+1} = \alpha_t + \gamma(\text{miscovered} - \alpha)$, which is the inverse of the Gibbs–Candès update rule. Under miscovering, α_t grew (interval narrowed) when it should have shrunk (interval widened). Coverage collapsed to 0.042. The fix reverses the sign to match Equation (1): $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \text{miscovered})$. After the fix, coverage rose to 0.895 on the $3\times$ drift stream.

TABLE III
PER-FLOOR SCORE PROGRESSION: INITIAL RUN VS. POST-FIX V0.1.

Floor	Initial run	Post-fix v0.1
HF-8	PASS	PASS
HF-9	PASS	PASS
HF-10	FAIL (IndexError)	PASS-WITH-CAVEAT
HF-12	FAIL (depended on HF-10)	PASS-WITH-CAVEAT
HF-13	PASS	PASS
HF-14	PASS	PASS
HF-15 (struct.)	PASS	PASS
Drift	FAIL (ACI 0.042)	PASS

E. Score progression

Table III shows the score progression. Both failures were genuine architectural defects, not benchmark misalignments; both were fixed without modifying any benchmark threshold. This is the intended behavior of ZCS-6 Phase 5: the first runnable version may fail, the failure is informative, the fix is at the architecture level, the benchmark does not move.

VIII. DISCUSSION

A. Threats to validity

The v0.1 results are subject to several threats to validity. First, the rule-based inference layer means the HF-12 autonomous-resolution rate measurement is not a true test of the gate—it measures only that the pipeline runs end-to-end without exception. A real LLM inference layer (Mistral-7B via Ollama, planned for v0.2) will produce confidence distributions that exercise ACI gating in a way the v0.1 stub cannot.

Second, the drift clause evaluation covers one of five named distributions. The chosen distribution (a scalar regression analog of crew metabolic rate) is the easiest case: Gaussian noise with growing variance. The other four distributions—thermal load, CO_2 profile, dust accumulation, bearing wear—have different drift structures, including monotonic accumulation patterns that may stress ACI under harder regimes.

Third, the hardware-isolation surrogate is a software contract, not a hardware mechanism. The v0.1 penetration test (1,000 ML-impersonation attempts all denied) confirms the software contract holds; it does not confirm silicon-level enforcement.

Fourth, the formal-verification status of the Mercury kernel is informal. Theorem 3 is a paragraph-length proof sketch, not a Lean or Coq proof. Production deployment would require machine-checkable proofs.

B. Trivial-pass surfaces in v1.1

VBX-ISPS-BENCH-v1.1 contains five known trivial-pass surfaces flagged for Phase 6 vigilance. We summarize them here for completeness:

- 1) HF-12 + HF-9 composition: a system that reclassifies actions out of `AUTONOMOUS_ELIGIBLE` to evade HF-12 (sandbagging). The counter-mechanism is that classification distributions are themselves audited.

- 2) HF-1 leaf-level gaming: aggressive leaf reliability assignments that compose to pass without supplier evidence. Counter: supplier evidence required at procurement.
- 3) HF-2 extrapolation overreach: extrapolation rationale claims confidence beyond what the test data supports. Counter: uncertainty bounds required and reviewed.
- 4) HF-13 chain-signing throttle: delaying chain signing to keep per-event latency low while violating throughput. Counter: throughput audited under HF-8.
- 5) HF-15 evidence package volume cheat: a long binder with shallow mapping. Counter: reviewer sign-off requires clause-to-floor specificity.

These are not v1.1 failures; they are surfaces Phase 6 must watch.

C. Comparison with reference systems

VBX-ISPS-BENCH-v1.1 was scored against four reference systems: EMU, xEMU/AxEMU, Z-2 with Aletheia DAC bolt-on (hypothetical), and xEMU with Aletheia and MVCI bolt-on (hypothetical). No reference system passes the v1.1 benchmark. The closest hypothetical (Z-2 + Aletheia) fails six floors, primarily mission-class shell floors (HF-1, HF-3, HF-6) and the radiation/dust/multi-mode floors that no current demonstrator has flight-validated. The baselines confirm the whitespace claim made in Section I: the substrate convergence is genuinely unsolved across current and announced suit programs.

IX. FUTURE WORK

A. v0.2 substrate closure

Seven items constitute the v0.2 backlog, derived from the v0.1 gap analysis:

- 1) Integrate Mistral-7B inference via Ollama, replacing the rule-based stub. Expected outcome: HF-12 autonomous-resolution rate drops from 1.000 to a meaningful sub-1.000 value, exercising ACI gating.
- 2) Produce Lean 4 or Coq machine-checkable proofs of the Mercury subleq emulator semantics and the O₂ control loop. The subleq ISA's minimality makes this tractable.
- 3) Specify a hardware-architecture document for silicon-level safety-bus enforcement, with a candidate path via the Mercury AWS F1 CL wrapper.
- 4) Extend the drift suite to all five named distributions, including monotonic accumulation patterns for dust and bearing wear.
- 5) Produce a production-grade scenario library of ~10,000 stratified events drawn from analog mission data, hand-crafted adversarial cases, and where available, declassified Apollo/Shuttle/ISS EVA telemetry.
- 6) Engage a qualified actuarial underwriter for the HF-15 dry-run.
- 7) Produce the Robot AIBOM per OMB M-25-22 Annex C, tied to the active 90-day AIBOM generator workstream at Visionblox.

B. Phase 6: vertical-integration attack

Phase 6 of ZCS-6 attacks the solution recursively until no seam survives. For v0.1, Phase 6 must include: (a) seam analysis at every layer boundary (Aletheia–MVCI, MVCI–Mercury, MVCI–Civium, all–bus), (b) out-of-distribution behavior beyond the scenario library, (c) freedom-to-operate re-check against current spacesuit patents and embodied-AI compliance patents, (d) compliance attack against the standards mapping, and (e) bench-maxing self-audit on the benchmark itself.

C. v1.2 shell scope

The eight shell-scope floors require industrial test facilities. Engagement with a hardware partner triggers v1.2 of the benchmark, in which facility-specific verification protocols are committed (which chamber, which beam line, which suit-port design). The substrate is decoupled from this critical path by construction: v0.x continues independently.

X. CONCLUSION

We argued that the unsolved engineering surface for an interplanetary spacesuit is substrate convergence—autonomous compliance-graded decision-making with provenance, deterministic kernel, drift-aware confidence, and embodied-AI compliance evidence—not shell mechanics. We presented a falsification-first methodology (ZCS-6) that cryptographically commits a benchmark before constructing the solution, and applied it to a fifteen-floor hard-fail benchmark (VBX-ISPS-BENCH-v1.1) scope-partitioned into substrate and shell. We built a substrate prototype (v0.1) that passes 31/31 tests against the seven substrate-scope floors, with five outright passes, two passes with documented caveats, and zero failures. Two architectural defects surfaced during evaluation and were fixed without softening any benchmark threshold. We provided an honest gap analysis enumerating what v0.1 does not yet do and what v0.2 must add.

The substrate is the IP moat; the shell is engineering. Substrate convergence is buildable on commodity hardware now. The falsification-first methodology is the discriminator from prior work in autonomous life-support: the benchmark precedes the solution, revisions are versioned, and the score progression is auditable.

REPRODUCIBILITY AND ARTIFACT AVAILABILITY

The substrate prototype source, test suite, benchmark JSON, delta record, baseline scan, chain commit manifests, and evidence package are committed to the chain as VBX-ISPS-LEDGER#0001 through VBX-ISPS-LEDGER#0003. The substrate code runs on Python 3.12 with three open-source dependencies. All numerical results in Section VII are reproducible by executing `pytest tests/` against the substrate v0.1 source tree.

ACKNOWLEDGMENT

This work was conducted as a ZCS-6 Phase 5 deliverable at Visionblox LLC / Zuup Innovation Lab. The author thanks Anthropic’s Claude Opus 4.7 for ZCS-6 methodology execution support, including Phase 3 attack identification and Phase 5 architecture instantiation.

REFERENCES

- [1] NASA, *Exploration Extravehicular Mobility Unit (xEMU) Architecture*. NASA Technical Memorandum, 2020.
- [2] —, “NASA Selects Axiom Space to Produce Moonwalking System for Artemis Missions,” NASA Press Release 22-091, 2022.
- [3] A. Ross, R. Rhodes, and D. Graziosi, “Z-2 advanced spacesuit demonstrator: Test results and lessons learned,” NASA Johnson Space Center, Tech. Rep., 2014.
- [4] K. S. Thomas and H. J. McMann, “U.s. spacesuits,” *Springer-Praxis*, 2006.
- [5] I. P. Abramov and A. I. Skoog, *Russian Spacesuits*. Springer-Praxis, 2003.
- [6] A. J. Ross, “Z-2 suit design,” NASA Johnson Space Center, Tech. Rep., 2014.
- [7] NASA Johnson Space Center, “Emu reference manual,” NASA, Tech. Rep., 2018.
- [8] J. T. Sutton and S. Casaburi, “Decompression sickness in eva operations: A 30-year retrospective,” *Aerospace Medicine and Human Performance*, 2018.
- [9] F. A. Korona, “xemu subsystem design overview,” NASA Johnson Space Center, Tech. Rep., 2020.
- [10] B. A. Jones, “Z-1 prototype suit testing,” NASA Glenn Research Center, Tech. Rep., 2012.
- [11] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, “sel4: Formal verification of an os kernel,” in *Proc. ACM SOSP*, 2009.
- [12] X. Leroy, “Formal verification of a realistic compiler,” *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.
- [13] P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, “The ASTRÉE analyzer,” in *Proc. ESOP*, 2005.
- [14] N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin, “Dependent types and monadic effects in F*,” in *Proc. ACM POPL*, 2016.
- [15] L. de Moura and S. Ullrich, “The lean 4 theorem prover and programming language,” in *Proc. CADE*, 2021.
- [16] The mathlib Community, “The Lean mathematical library,” *Proc. ACM CPP*, 2020.
- [17] F. Mavaddat and B. Parhami, “URISC: The ultimate reduced instruction set computer,” *International Journal of Electrical Engineering Education*, vol. 25, no. 4, pp. 327–334, 1988.
- [18] S. Nürnberger and M. Backes, “Single-instruction set computers in hardware security,” in *Workshop on Trustworthy Embedded Devices*, 2015.
- [19] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.
- [20] G. Shafer and V. Vovk, “A tutorial on conformal prediction,” *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.
- [21] I. Gibbs and E. J. Candès, “Adaptive conformal inference under distribution shift,” *Advances in Neural Information Processing Systems*, 2021.
- [22] A. Bhatnagar, H. Wang, C. Xiong, and Y. Bai, “Improved online conformal prediction via strongly adaptive online learning,” in *Proc. ICML*, 2023.
- [23] M. Zaffran, O. Féron, Y. Goude, J. Josse, and A. Dieuleveut, “Adaptive conformal predictions for time series,” in *Proc. ICML*, 2022.
- [24] A. K. Wooden, “Aletheia DAC: Drift-aware claim substrate with adaptive conformal inference,” Visionblox/Zuup Innovation Lab internal report, 2026.
- [25] International Organization for Standardization, “ISO 12100: Safety of machinery—general principles for design—risk assessment and risk reduction,” ISO Standard, 2010.
- [26] Robotic Industries Association, “ANSI/RIA r15.08-1: Industrial mobile robots—safety requirements,” ANSI Standard, 2020.
- [27] Association for Advancing Automation, “ANSI/A3 r15.06-2025: Safety requirements for industrial robots and robot systems,” ANSI Standard, 2025.
- [28] —, “ANSI/A3 r15.08-2: Autonomy levels for mobile robots,” ANSI Standard, 2023.
- [29] Office of Management and Budget, “OMB memorandum m-25-22: Federal ai use and ai bill of materials requirements,” Federal Memorandum, 2025.
- [30] U.S. Department of Health and Human Services, “HIPAA security rule: 45 CFR part 164, subpart C,” Federal Regulation, 2013.
- [31] NASA, “NASA-STD-3001 vol. 2: Human integration design handbook—crew health and performance,” NASA Technical Standard, 2024.
- [32] B. Laurie, “Certificate transparency,” in *Communications of the ACM*, 2014.
- [33] P. Todd, “Opentimestamps: Scalable, trust-minimized, distributed timestamping with Bitcoin,” opentimestamps.org, 2018.
- [34] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [35] C. A. E. Goodhart, “Problems of monetary management: The U.K. experience,” *Papers in Monetary Economics*, 1975.